Small Moduli, Big Impact: Toward Communication-Efficient Lattice-Based OPRFs

Lena Heimberger

Monash University

1st of December, 2025

Hi!

- PhD student @Graz University of Technology
  Supervisor: Christian Rechberger

Hi!

- PhD student @Graz University of Technology
  Supervisor: Christian Rechberger
- I study lattice-based OPRFs from heuristic assumptions to get efficient OPRFs.

Hi!

- PhD student @Graz University of Technology
  Supervisor: Christian Rechberger
- I study lattice-based OPRFs from heuristic assumptions to get efficient OPRFs.
- Recently built an OPRF more efficient than elliptic curves! [HKL+25]

Hi!

- PhD student @Graz University of Technology
  Supervisor: Christian Rechberger
- I study lattice-based OPRFs from heuristic assumptions to get efficient OPRFs.
- Recently built an OPRF more efficient than elliptic curves! [HKL+25]
- Needs more analysis:
  - Lower bounds: How big does the LWR modulus have to be in theory?

Hi!

- PhD student @Graz University of Technology
  Supervisor: Christian Rechberger
- I study lattice-based OPRFs from heuristic assumptions to get efficient OPRFs.
- Recently built an OPRF more efficient than elliptic curves! [HKL+25]
- Needs more analysis:
  - Lower bounds: How big does the LWR modulus have to be in theory?
  - Concrete lattice attacks on small-moduli instantiations

Hi!

- PhD student @Graz University of Technology
  Supervisor: Christian Rechberger
- I study lattice-based OPRFs from heuristic assumptions to get efficient OPRFs.
- Recently built an OPRF more efficient than elliptic curves! [HKL+25]
- Needs more analysis:
  - Lower bounds: How big does the LWR modulus have to be in theory?
  - Concrete lattice attacks on small-moduli instantiations
  - underlying PRF: can we find differentials within small-moduli PRFs?
    joint work with Sonja Denk (Master student @ TUG)

Hi!

- PhD student @Graz University of Technology
  Supervisor: Christian Rechberger
- I study lattice-based OPRFs from heuristic assumptions to get efficient OPRFs.
- Recently built an OPRF more efficient than elliptic curves! [HKL+25]
- Needs more analysis:
  - Lower bounds: How big does the LWR modulus have to be in theory?
  - Concrete lattice attacks on small-moduli instantiations
  - underlying PRF: can we find differentials within small-moduli PRFs?
    joint work with Sonja Denk (Master student @ TUG)
  - efficient preprocessing from PQ-OT without KEM

Hi!

- PhD student @Graz University of Technology
  Supervisor: Christian Rechberger
- I study lattice-based OPRFs from heuristic assumptions to get efficient OPRFs.
- Recently built an OPRF more efficient than elliptic curves! [HKL+25]
- Needs more analysis:
  - Lower bounds: How big does the LWR modulus have to be in theory?
  - Concrete lattice attacks on small-moduli instantiations
  - underlying PRF: can we find differentials within small-moduli PRFs?
    joint work with Sonja Denk (Master student @ TUG)
  - efficient preprocessing from PQ-OT without KEM
  - verifiable Naor-Reingold PRFs and consistency proofs

Graz, Austr$_{al}$ia, Republic of

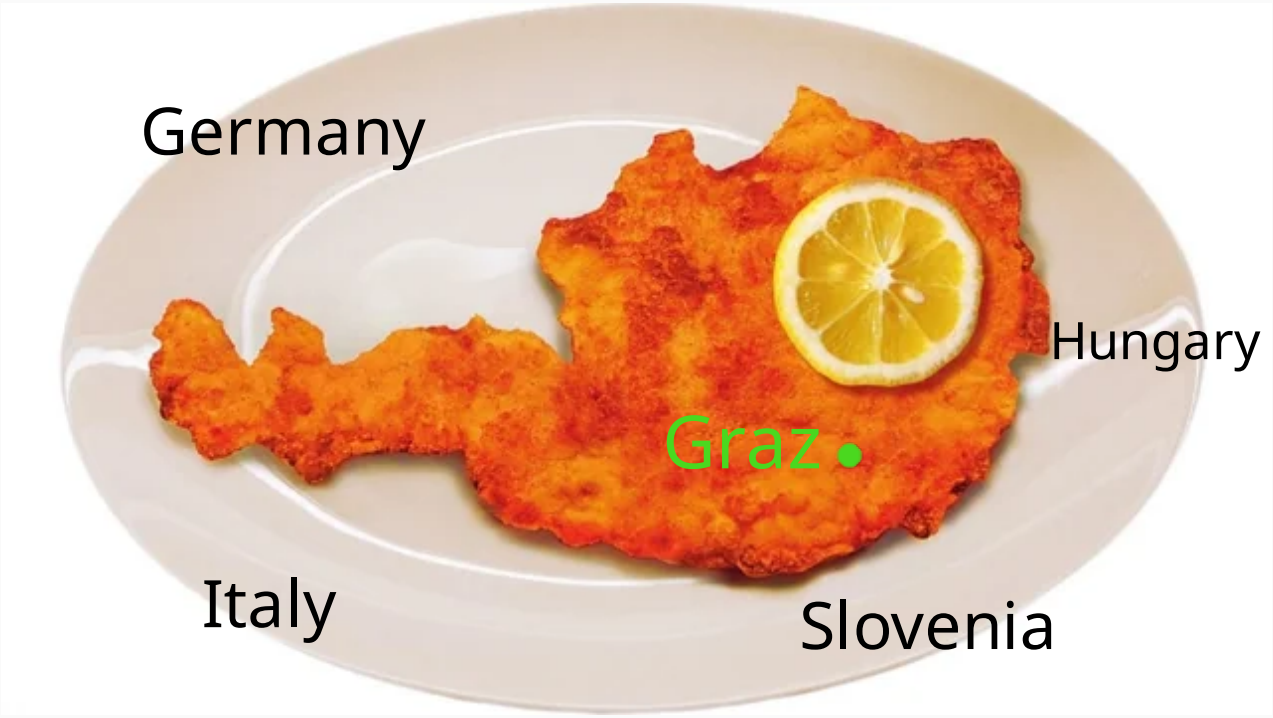


Figure 1: Schnitzel map from `https://www.reddit.com/r/mapporncirclejerk/comments/e3x9h4/map_of_austria_but_its_actually_a_schnitzel/`, modified by me.

Motivation for Oblivious Pseudorandom Functions

- simple(r) primitive from asymmetric cryptography

Motivation for Oblivious Pseudorandom Functions

- simple(r) primitive from asymmetric cryptography
- gives rise to a number of cool constructions
- cheap!

Motivation for Oblivious Pseudorandom Functions

- simple(r) primitive from asymmetric cryptography
- gives rise to a number of cool constructions
- cheap! ...pre-quantum

Pseudorandom Function

A pseudorandom function (PRF) [GGM84, GGM86] is a deterministic and polynomial time function $F\colon \{0,1\}^k \times \{0,1\}^x \to \{0,1\}^y$ such that there is no probabilistic polynomial-time algorithm to distinguish any output y from a randomly chosen element from $\{0,1\}^y$.

Definitions

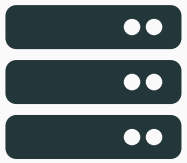
Client

input $c \in \mathcal{C}$, gets y

compute

$$y := F_k(c)$$

Server



key $k \in \mathcal{K}$

Oblivious Pseudorandom Function

An oblivious pseudorandom function (OPRF) [FIPR05] is a protocol between two parties. One party holds the secret key k and the other holds their secret input x. The OPRF privately realizes the joint computation outputting $F_k(x)$ for a PRF F to the party holding x, and nothing to the party holding k.

Additional Properties

Partial Obliviousness In a POPRF, the client submits a tag t in addition to the input x and gets the evaluation $y := F_k(t, x)$. t contains additional information, e.g. a validity period.

Additional Properties

Partial Obliviousness In a POPRF, the client submits a tag t in addition to the input x and gets the evaluation $y := F_k(t, x)$. t contains additional information, e.g. a validity period.

Threshold Get a valid evaluation from a t-out-of-n servers.

Additional Properties

Partial Obliviousness In a POPRF, the client submits a tag t in addition to the input x and gets the evaluation $y := F_k(t, x)$. t contains additional information, e.g. a validity period.

Threshold Get a valid evaluation from a t-out-of-n servers.

Verifiability The client can ensure that a pre-commited server key k was used to compute $y = F_k(x)$.
VOPRF+POPRF=VPOPRF

Additional Properties

Partial Obliviousness In a POPRF, the client submits a tag t in addition to the input x and gets the evaluation $y := F_k(t, x)$. t contains additional information, e.g. a validity period.

Threshold Get a valid evaluation from a t-out-of-n servers.

Verifiability The client can ensure that a pre-commited server key k was used to compute $y = F_k(x)$.
VOPRF+POPRF=VPOPRF

Public Verifiability means we built a blind signature.

How would an ideal OPRF look like?

Security. ideally malicious security, i.e. a malicious adversary cannot learn anything they shouldn't

How would an ideal OPRF look like?

Security. ideally malicious security, i.e. a malicious adversary cannot learn anything they shouldn't (doesn't imply detection)

How would an ideal OPRF look like?

Security. ideally malicious security, i.e. a malicious adversary cannot learn anything they shouldn't (doesn't imply detection)

Round-Optimality. A round is a period where all parties can exchange messages simultaneously.

How would an ideal OPRF look like?

Security. ideally malicious security, i.e. a malicious adversary cannot learn anything they shouldn't (doesn't imply detection)

Round-Optimality. A round is a period where all parties can exchange messages simultaneously.
A non-interactive protocol (e.g. NIZK): single round,
interactive protocol: two rounds

How would an ideal OPRF look like?

Security. ideally malicious security, i.e. a malicious adversary cannot learn anything they shouldn't (doesn't imply detection)

Round-Optimality. A round is a period where all parties can exchange messages simultaneously.
A non-interactive protocol (e.g. NIZK): single round, interactive protocol: two rounds

Communication size. Elliptic curve OPRFs: 766 bits of communication. Best lattice-based: tens of kilobytes, in a semi-honest setting and with preprocessing.

How would an ideal OPRF look like?

Security. ideally malicious security, i.e. a malicious adversary cannot learn anything they shouldn't (doesn't imply detection)

Round-Optimality. A round is a period where all parties can exchange messages simultaneously.
A non-interactive protocol (e.g. NIZK): single round, interactive protocol: two rounds

Communication size. Elliptic curve OPRFs: 766 bits of communication. Best lattice-based: tens of kilobytes, in a semi-honest setting and with preprocessing.

Computational complexity. Generic lattices are fast, and we can speed them up further using vector instructions or even GPUs.

How would an ideal OPRF look like?

Security. ideally malicious security, i.e. a malicious adversary cannot learn anything they shouldn't (doesn't imply detection)

Round-Optimality. A round is a period where all parties can exchange messages simultaneously.
A non-interactive protocol (e.g. NIZK): single round, interactive protocol: two rounds

Communication size. Elliptic curve OPRFs: 766 bits of communication. Best lattice-based: tens of kilobytes, in a semi-honest setting and with preprocessing.

Computational complexity. Generic lattices are fast, and we can speed them up further using vector instructions or even GPUs. Kyber768 takes between a third to half of the computational time of ECDH Curve 25519!

Anything else?

Preprocessing. Repeated computations? Shift data-independent communication and computation to a preprocessing phase.

Anything else?

Preprocessing. Repeated computations? Shift data-independent communication and computation to a preprocessing phase. Fully online protocols: online variant or perform the precomputation on the fly.

Anything else?

Preprocessing. Repeated computations? Shift data-independent communication and computation to a preprocessing phase. Fully online protocols: online variant or perform the precomputation on the fly.

Trusted Setup. In addition to preprocessing, some OPRFs rely on a trusted setup.

Anything else?

Preprocessing. Repeated computations? Shift data-independent communication and computation to a preprocessing phase. Fully online protocols: online variant or perform the precomputation on the fly.

Trusted Setup. In addition to preprocessing, some OPRFs rely on a trusted setup.But: trust is a hard problem in itself, and sometimes a trusted setup just isn't available (e.g. for CSIDH).

OPAQUE

	Client (pwd)		**Serve**
1. Registration:	$k' = F_k(pwd)$	$\xleftrightarrow{OPRF}$	
	Sample (pk, sk)	$\xrightarrow{pk,\, ct = Enc_{k'}(sk)}$	Store $(pk$

OPAQUE

	Client (pwd)		**Serve**
1. Registration:	$k' = F_k(pwd)$	$\xleftrightarrow{OPRF}$	
	Sample (pk, sk)	$\xrightarrow{pk,\, ct = Enc_{k'}(sk)}$	Store $(pk$
2. Login:		$\xleftarrow{ct}$	
	$k' = F_k(pwd)$	$\xleftrightarrow{OPRF}$	
	$sk = Dec_{k'}(ct)$	$\xleftrightarrow{\text{AKE using } sk}$	

OPAQUE

	Client (pwd)		**Serve**
1. Registration:	$k' = F_k(pwd)$	$\xleftrightarrow{OPRF}$	
	Sample (pk, sk)	$\xrightarrow{pk,\, ct = Enc_{k'}(sk)}$	Store $(pk$
2. Login:		$\xleftarrow{ct}$	
	$k' = F_k(pwd)$	$\xleftrightarrow{OPRF}$	
	$sk = Dec_{k'}(ct)$	$\xleftrightarrow{\text{AKE using } sk}$	

Which properties are important?

OPAQUE: Properties

- doesn't reveal output

OPAQUE: Properties

- doesn't reveal output
- server still passively observes the success or failure of an authentication attempt

OPAQUE: Properties

- doesn't reveal output
- server still passively observes the success or failure of an authentication attempt
- OPRF must be secure against a malicious server

OPAQUE: Properties

- doesn't reveal output
- server still passively observes the success or failure of an authentication attempt
- OPRF must be secure against a malicious server
- Verifiability?

Private Set Intersection (PSI)

OPRFs are useful for unbalanced sets, where one party, typically the server in client-server protocols, has a significantly larger set than the other party.

Alice $(x_1, \ldots, x_m)$ **Bob** $(y_1, \ldots, y_n), k$

$\{F_k(x_i)\}_{i \in [m]}$

$$\xleftrightarrow{\quad OPRF \quad}$$

$$\xleftarrow{\quad \{z_j = F_k(y_i)\}_{j \in [n]} \quad}$$

$F_k(x_i) = z_j$. Otherwise $F_k(y_j)$ is pseudorandom.

Private Set Intersection (PSI)

OPRFs are useful for unbalanced sets, where one party, typically the server in client-server protocols, has a significantly larger set than the other party.

Alice $(x_1, \ldots, x_m)$ **Bob** $(y_1, \ldots, y_n), k$

$\{F_k(x_i)\}_{i \in [m]}$

$$\xleftrightarrow{\quad OPRF \quad}$$

$$\xleftarrow{\quad \{z_j = F_k(y_i)\}_{j \in [n]} \quad}$$

$F_k(x_i) = z_j$. Otherwise $F_k(y_j)$ is pseudorandom.

Which properties are important?

PSI: Properties

- per-client keys may cause segmentation

PSI: Properties

- per-client keys may cause segmentation
- semi-honest server may be realistic:

PSI: Properties

- per-client keys may cause segmentation
- semi-honest server may be realistic:
  - regulatory and reputational incentives [KRS+19]

PSI: Properties

- per-client keys may cause segmentation
- semi-honest server may be realistic:
  - regulatory and reputational incentives [KRS+19]
  - inherent trust
    government agencies identifying duplicate entries

PSI: Properties

- per-client keys may cause segmentation
- semi-honest server may be realistic:
  - regulatory and reputational incentives [KRS+19]
  - inherent trust
    government agencies identifying duplicate entries
  - anywhere where PSI is applied to limit the risk of data breaches

PSI: Properties

- per-client keys may cause segmentation
- semi-honest server may be realistic:
  - regulatory and reputational incentives [KRS+19]
  - inherent trust
    government agencies identifying duplicate entries
  - anywhere where PSI is applied to limit the risk of data breaches
- PSI can be a good base case for non-traditional OPRFs

Naor-Reingold and Lattices

Naor-Reingold PRF [NR04]

- The Naor-Reingold PRF [NR04] that generically builds a PRF from an Abelian group action $*$.

Naor-Reingold PRF [NR04]

- The Naor-Reingold PRF [NR04] that generically builds a PRF from an Abelian group action $*$.
- To compute the PRF over m input bits, we sample $m + 1$ group elements $[g_0, g_1, \ldots, g_m]$.

Naor-Reingold PRF [NR04]

- The Naor-Reingold PRF [NR04] that generically builds a PRF from an Abelian group action $*$.
- To compute the PRF over m input bits, we sample $m + 1$ group elements $[g_0, g_1, \ldots, g_m]$.
- Start with group action with g_0.

Naor-Reingold PRF [NR04]

- The Naor-Reingold PRF [NR04] that generically builds a PRF from an Abelian group action $*$.
- To compute the PRF over m input bits, we sample $m + 1$ group elements $[g_0, g_1, \ldots, g_m]$.
- Start with group action with g_0.
- For every following group element, the group action is performed when the ith bit of the input is set.

Naor-Reingold PRF [NR04]

- The Naor-Reingold PRF [NR04] that generically builds a PRF from an Abelian group action $*$.
- To compute the PRF over m input bits, we sample $m + 1$ group elements $[g_0, g_1, \ldots, g_m]$.
- Start with group action with g_0.
- For every following group element, the group action is performed when the ith bit of the input is set.
- Automatically hashes to group!

Naor-Reingold PRF [NR04]

- The Naor-Reingold PRF [NR04] that generically builds a PRF from an Abelian group action $*$.
- To compute the PRF over m input bits, we sample $m+1$ group elements $[g_0, g_1, \ldots, g_m]$.
- Start with group action with g_0.
- For every following group element, the group action is performed when the ith bit of the input is set.
- Automatically hashes to group!

Example (Naor-Reingold with seven input bits)

For example, for 0110111 we would compute the group action $g_0 * g_2 * g_3 * g_5 * g_6 * g_7$.

Naor-Reingold PRF [NR04]

- The Naor-Reingold PRF [NR04] that generically builds a PRF from an Abelian group action $*$.
- To compute the PRF over m input bits, we sample $m+1$ group elements $[g_0, g_1, \ldots, g_m]$.
- Start with group action with g_0.
- For every following group element, the group action is performed when the ith bit of the input is set.
- Automatically hashes to group!

Example (Naor-Reingold with seven input bits)

For example, for 0110111 we would compute the group action $g_0 * g_2 * g_3 * g_5 * g_6 * g_7$.

Good framework to study group actions!

Learning with Rounding PRFs

- derandomize Learning with errors

Learning with Rounding PRFs

- derandomize Learning with errors
  - SIS: $\mathbf{As} = \mathbf{x}$
  - LWE: add noise to get approximate solution $\mathbf{As} + \mathbf{e} = \mathbf{x}$
  - LWR: round from big modulus p to smaller modulus q
    $\lfloor \mathbf{As}_q \rceil_p$

Learning with Rounding PRFs

- derandomize Learning with errors
  - SIS: $\mathbf{As} = \mathbf{x}$
  - LWE: add noise to get approximate solution $\mathbf{As} + \mathbf{e} = \mathbf{x}$
  - LWR: round from big modulus p to smaller modulus q
    $\lfloor \mathbf{As}_q \rceil_p$
- NIST PKE/KEM Finalist: Saber [DKR+20]

Learning with Rounding PRFs

- derandomize Learning with errors
  - SIS: $\mathbf{As} = \mathbf{x}$
  - LWE: add noise to get approximate solution $\mathbf{As} + \mathbf{e} = \mathbf{x}$
  - LWR: round from big modulus p to smaller modulus q $\lfloor \mathbf{As}_q \rceil_p$
- NIST PKE/KEM Finalist: Saber [DKR+20]
- Naor-Reingold PRF from LWR:

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\mathbf{k}_0 \prod_{i=1}^{m} \mathbf{k}_i^{c_i}\right)$$

Spring is a lattice-based PRF

- 2014: heuristic LWR PRF with small modulus

Spring is a lattice-based PRF

- 2014: heuristic LWR PRF with small modulus
- two modes:

Spring is a lattice-based PRF

- 2014: heuristic LWR PRF with small modulus
- two modes:
  - SPRING-BCH with $q = 257$

Spring is a lattice-based PRF

- 2014: heuristic LWR PRF with small modulus
- two modes:
  - $\textsc{Spring}$-BCH with $q = 257$
  - $\textsc{Spring}$-CRT with $q = 514$

Spring is a lattice-based PRF

- 2014: heuristic LWR PRF with small modulus
- two modes:
  - $\textsc{Spring}$-BCH with $q = 257$
  - $\textsc{Spring}$-CRT with $q = 514$
- [HKL+25] focus in $\textsc{Spring}$-BCH: rounding bias reduction using extended BCH code $[128, 64, 22]$

Spring is a lattice-based PRF

- 2014: heuristic LWR PRF with small modulus
- two modes:
  - $\textsc{Spring}$-BCH with $q = 257$
  - $\textsc{Spring}$-CRT with $q = 514$
- [HKL+25] focus in $\textsc{Spring}$-BCH: rounding bias reduction using extended BCH code $[128, 64, 22]$

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\mathbf{k}_0 \prod_{i=1}^{m} \mathbf{k}_i^{c_i}\right)$$

Spring is a lattice-based PRF

- 2014: heuristic LWR PRF with small modulus
- two modes:
  - Spring-BCH with $q = 257$
  - Spring-CRT with $q = 514$
- [HKL+25] focus in Spring-BCH: rounding bias reduction using extended BCH code $[128, 64, 22]$

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\mathrm{iNTT}\left(\mathbf{k}_0 \odot \prod_{i=1}^{m} \mathbf{k}_i^{c_i}\right)\right)$$

Spring is a lattice-based PRF

- 2014: heuristic LWR PRF with small modulus
- two modes:
  - $\textsc{Spring}$-BCH with $q = 257$
  - $\textsc{Spring}$-CRT with $q = 514$
- [HKL+25] focus in $\textsc{Spring}$-BCH: rounding bias reduction using extended BCH code $[128, 64, 22]$

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\mathrm{iNTT}\left(\mathrm{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

Multiplication is addition of exponents!

$$g^a g^b = g^{a+b} \text{ for some generator } g$$

Why $q = \{257\}$?

- Choice of modulus from ease of implementation:
- $q = 257$
  - only use invertable polynomials

Why $q = \{257\}$?

- Choice of modulus from ease of implementation:
- $q = 257$
  - only use invertable polynomials
  - Computation mod q: use units (nonzero coefficients)

Why $q = \{257\}$?

- Choice of modulus from ease of implementation:
- $q = 257$
  - only use invertable polynomials
  - Computation mod q: use units (nonzero coefficients) avoid zeroing out
  - Fermat's Theorem for prime q:

$$g^{q-1} \equiv g^0 \equiv 1 \mod q$$

Why $q = \{257\}$?

- Choice of modulus from ease of implementation:
- $q = 257$
  - only use invertable polynomials
  - Computation mod q: use units (nonzero coefficients) avoid zeroing out
  - Fermat's Theorem for prime q:

$$g^{q-1} \equiv g^0 \equiv 1 \mod q$$

  - use subset-sum of exponents instead of multiplication

Why $q = \{257\}$?

- Choice of modulus from ease of implementation:
- $q = 257$
  - only use invertable polynomials
  - Computation mod q: use units (nonzero coefficients) avoid zeroing out
  - Fermat's Theorem for prime q:

$$g^{q-1} \equiv g^0 \equiv 1 \mod q$$

  - use subset-sum of exponents instead of multiplication
  - Sample in exponent form $\mod q - 1 = 256$

Why $q = \{257\}$?

- Choice of modulus from ease of implementation:
- $q = 257$
  - only use invertable polynomials
  - Computation mod q: use units (nonzero coefficients) avoid zeroing out
  - Fermat's Theorem for prime q:

$$g^{q-1} \equiv g^0 \equiv 1 \mod q$$

  - use subset-sum of exponents instead of multiplication
  - Sample in exponent form $\mod q - 1 = 256$ fits in a single byte!

Why $q = \{514\}$?

- Even modulus $q = 514 = 2 * 257$

Why $q = \{514\}$?

- Even modulus $q = 514 = 2 * 257$
- 257-component as on previous slide

Why $q = \{514\}$?

- Even modulus $q = 514 = 2 * 257$
- 257-component as on previous slide
- 2-component R_2^*; 257-style

Why $q = \{514\}$?

- Even modulus $q = 514 = 2 * 257$
- 257-component as on previous slide
- 2-component R_2^*; 257-style
  polynomial multiplication $\rightarrow$ coefficient multiplication $\rightarrow$ coefficient addition
  - polynomial is invertible when sum of coefficients is odd

Why $q = \{514\}$?

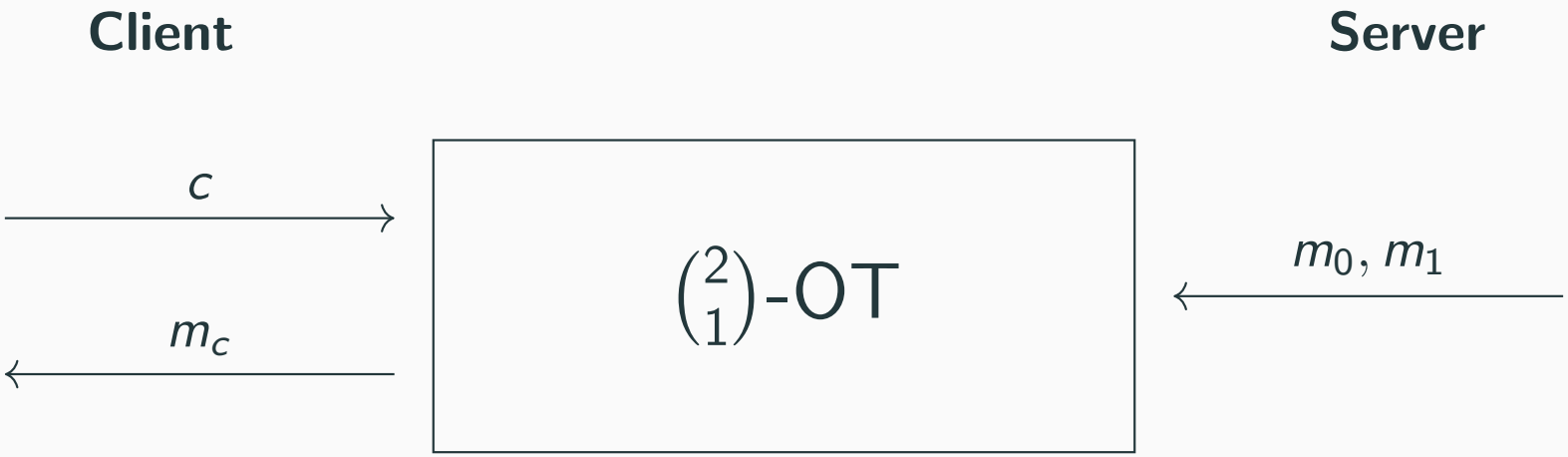
- Even modulus $q = 514 = 2 * 257$
- 257-component as on previous slide
- 2-component R_2^*; 257-style
  polynomial multiplication $\rightarrow$ coefficient multiplication $\rightarrow$ coefficient addition
  - polynomial is invertible when sum of coefficients is odd
  - Polynomial is represented in power representation:
    $A(x) = x_0 + b_1 x_1 + b_2 x_2 + ... + b_{127} x_{127}, b_i \in \{0, 1\}$

Why $q = \{514\}$?

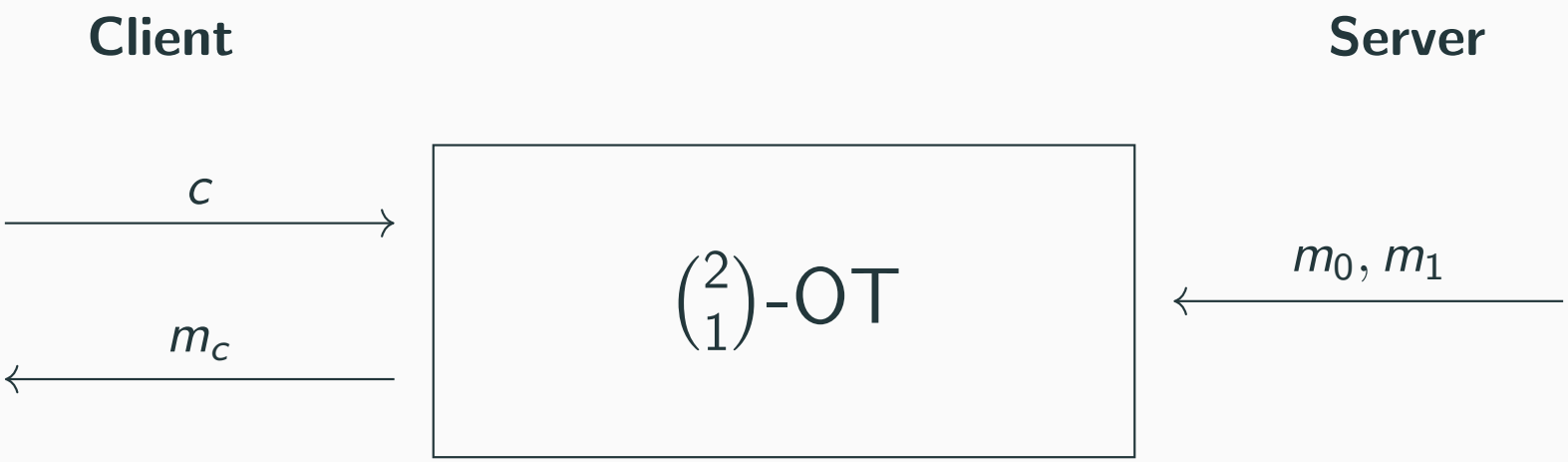
- Even modulus $q = 514 = 2 * 257$
- 257-component as on previous slide
- 2-component R_2^*; 257-style
  polynomial multiplication $\rightarrow$ coefficient multiplication $\rightarrow$ coefficient addition
  - polynomial is invertible when sum of coefficients is odd
  - Polynomial is represented in power representation:
    $A(x) = x_0 + b_1 x_1 + b_2 x_2 + ... + b_{127} x_{127}, b_i \in \{0, 1\}$
  - 2-component is evaluated similar to 257-component

Oblivious Evaluation

A little help from MPC friends

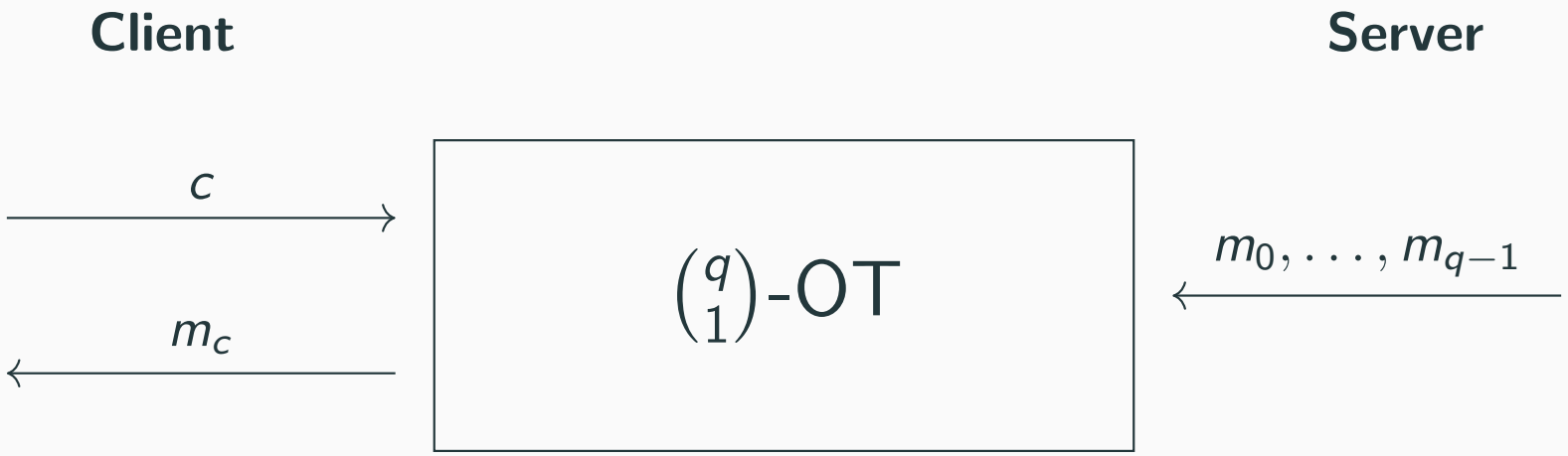


A little help from MPC friends

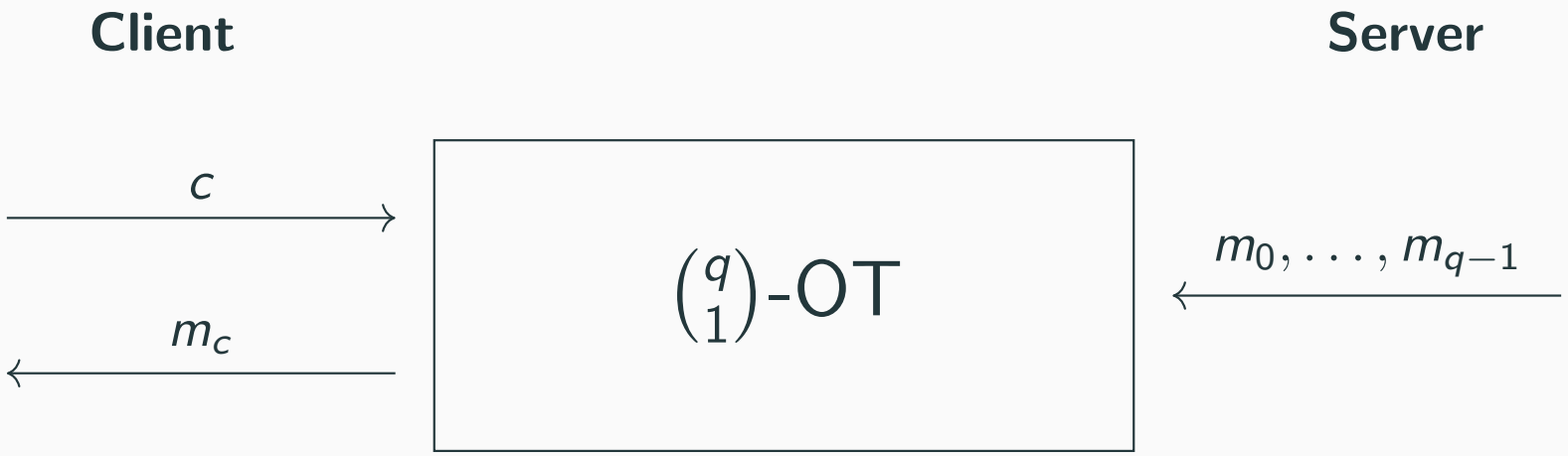


Extend with symmetric operations!

A little help from MPC friends (cont.)

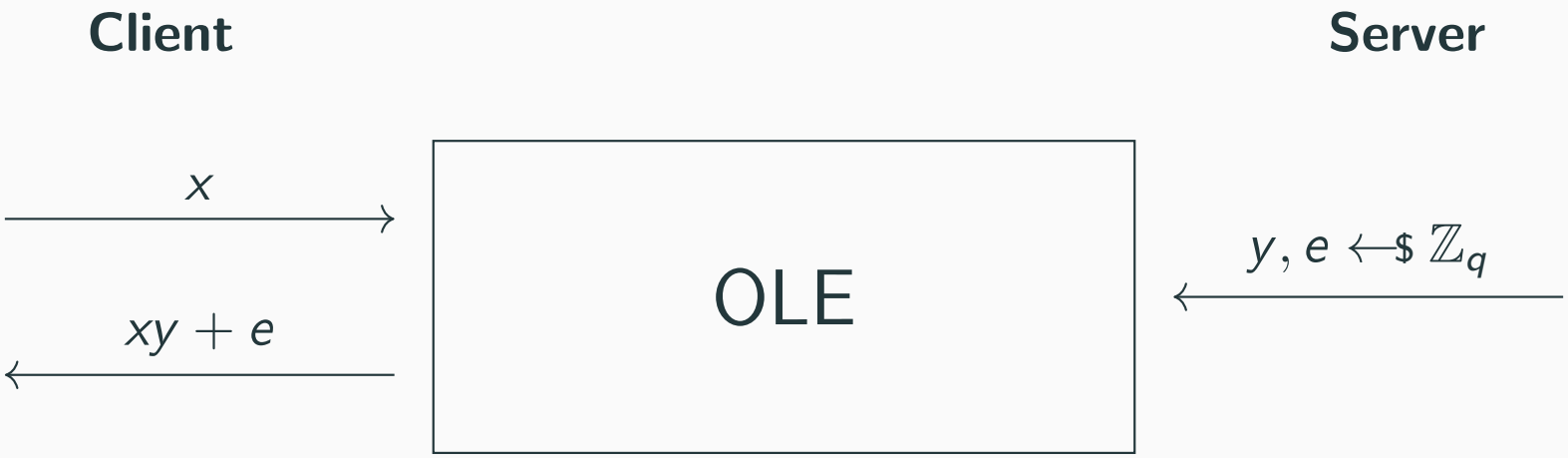


A little help from MPC friends (cont.)

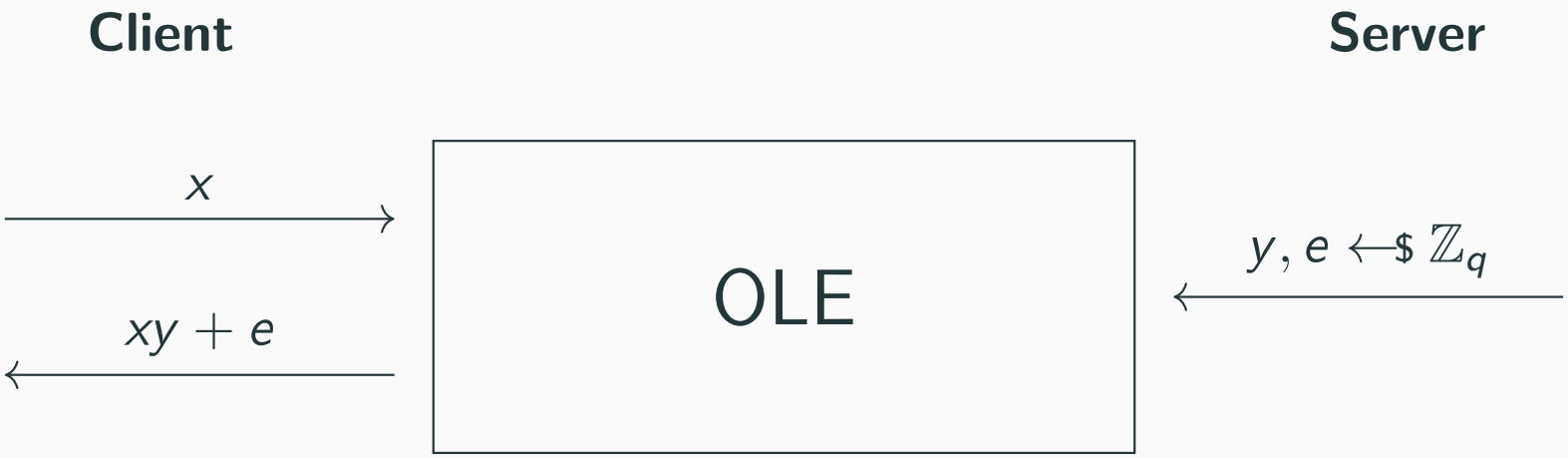


Generic construction from $\log q$ $\binom{2}{1}$-OTs !

A little help from MPC friends (cont.)



A little help from MPC friends (cont.)



Generic construction from $\log q$ $\binom{2}{1}$-OTs !

Leap

- Leap [HKL+25]: Naor-Reingold LWR OPRF!

Leap

- Leap [HKL+25]: Naor-Reingold LWR OPRF!
- very fast online: can shift almost everything to the preprocessing phase

Leap

- Leap [HKL+25]: Naor-Reingold LWR OPRF!
- very fast online: can shift almost everything to the preprocessing phase
- online phase takes less than a millisecond

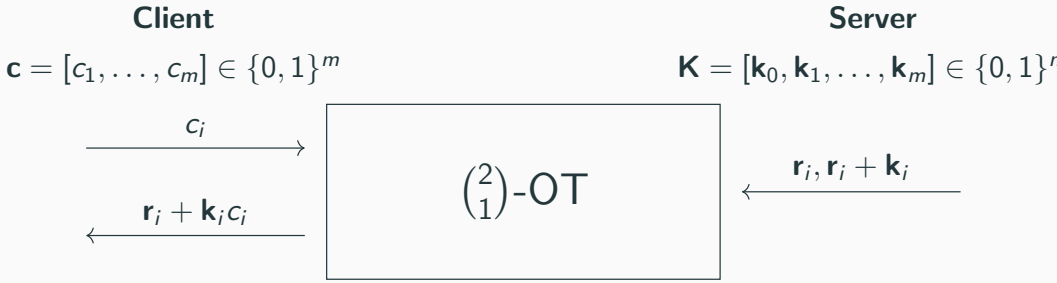
Leap

- Leap [HKL+25]: Naor-Reingold LWR OPRF!
- very fast online: can shift almost everything to the preprocessing phase
- online phase takes less than a millisecond
- symmetric-style evaluation

Leap

- Leap [HKL+25]: Naor-Reingold LWR OPRF!
- very fast online: can shift almost everything to the preprocessing phase
- online phase takes less than a millisecond
- symmetric-style evaluation
- optimized for PSI

An OPRF from Spring



$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

An OPRF from Spring

Client		Server
$\mathbf{c} = [c_1, \ldots, c_m] \in \{0,1\}^m$		$\mathbf{K} = [\mathbf{k}_0, \mathbf{k}_1, \ldots, \mathbf{k}_m] \in \{0,1\}^n$
$\xrightarrow{c_i}$	$\binom{2}{1}$-OT	$\xleftarrow{\mathbf{r}_i, \mathbf{r}_i + \mathbf{k}_i}$
$\xleftarrow{\mathbf{r}_i + \mathbf{k}_i c_i}$		
$\sum_{i=1}^{\kappa} \mathbf{r_i} + c_i \mathbf{k_i}$		$\mathbf{k}_0 - (\sum_{i=1}^{\kappa} \mathbf{r}_i)$

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

An OPRF from Spring

Client		**Server**
$\mathbf{c} = [c_1, \ldots, c_m] \in \{0,1\}^m$		$\mathbf{K} = [\mathbf{k}_0, \mathbf{k}_1, \ldots, \mathbf{k}_m] \in \{0,1\}^r$
$\xrightarrow{c_i}$	$\binom{2}{1}$-OT	$\xleftarrow{\mathbf{r}_i, \mathbf{r}_i + \mathbf{k}_i}$
$\xleftarrow{\mathbf{r}_i + \mathbf{k}_i c_i}$		
$\sum_{i=1}^{\kappa} \mathbf{r_i} + c_i \mathbf{k_i}$	♻	$\mathbf{k}_0 - (\sum_{i=1}^{\kappa} \mathbf{r}_i)$
$\mathbf{b} \leftarrow \prod_{i=1}^{\kappa} \mathbf{r_i} \mathbf{k_i}^{c_i}$		$\mathbf{a} \leftarrow \mathbf{k}_0 (\prod_{i=1}^{\kappa} \mathbf{r}_i)^{-1}$

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

An OPRF from Spring

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

Client		**Server**
$\mathbf{c} = [c_1, \ldots, c_m] \in \{0,1\}^m$		$\mathbf{K} = [\mathbf{k}_0, \mathbf{k}_1, \ldots, \mathbf{k}_m] \in \{0,1\}^n$
$\xrightarrow{c_i}$ $\xleftarrow{\mathbf{r}_i + \mathbf{k}_i c_i}$	$\binom{2}{1}$-OT	$\xleftarrow{\mathbf{r}_i, \mathbf{r}_i + \mathbf{k}_i}$
$\sum_{i=1}^{\kappa} \mathbf{r_i} + c_i \mathbf{k_i}$ $\mathbf{b} \leftarrow \prod_{i=1}^{\kappa} \mathbf{r_i} \mathbf{k_i}^{c_i}$	♻	$\mathbf{k}_0 - (\sum_{i=1}^{\kappa} \mathbf{r}_i)$ $\mathbf{a} \leftarrow \mathbf{k}_0 (\prod_{i=1}^{\kappa} \mathbf{r}_i)^{-1}$
$\xrightarrow{b_i}$ $\xleftarrow{y_i = a_i b_i + e_i}$	OLE	$\xleftarrow{a_i, e_i \leftarrow\$ \mathbb{Z}_q}$

An OPRF from Spring

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

Client		**Server**
$\mathbf{c} = [c_1, \ldots, c_m] \in \{0,1\}^m$		$\mathbf{K} = [\mathbf{k}_0, \mathbf{k}_1, \ldots, \mathbf{k}_m] \in \{0,1\}^n$
$\xrightarrow{c_i}$ $\xleftarrow{\mathbf{r}_i + \mathbf{k}_i c_i}$	$\binom{2}{1}$-OT	$\xleftarrow{\mathbf{r}_i, \mathbf{r}_i + \mathbf{k}_i}$
$\sum_{i=1}^{\kappa} \mathbf{r_i} + c_i \mathbf{k_i}$ $\mathbf{b} \leftarrow \prod_{i=1}^{\kappa} \mathbf{r_i} \mathbf{k_i}^{c_i}$	♻	$\mathbf{k}_0 - (\sum_{i=1}^{\kappa} \mathbf{r}_i)$ $\mathbf{a} \leftarrow \mathbf{k}_0 (\prod_{i=1}^{\kappa} \mathbf{r}_i)^{-1}$
$\xrightarrow{b_i}$ $\xleftarrow{y_i = a_i b_i + e_i}$	OLE	$\xleftarrow{a_i, e_i \leftarrow\$ \mathbb{Z}_q}$
$\mathbf{y} \leftarrow \sum_{i=0}^{n-1} (a_i \cdot b_i + e_i) X^i$		$\mathbf{e} \leftarrow \sum_{i=0}^{n-1} e_i X^i$

An OPRF from Spring

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

Client		**Server**
$\mathbf{c} = [c_1, \ldots, c_m] \in \{0,1\}^m$		$\mathbf{K} = [\mathbf{k}_0, \mathbf{k}_1, \ldots, \mathbf{k}_m] \in \{0,1\}^n$
$\xrightarrow{c_i}$ $\xleftarrow{\mathbf{r}_i + \mathbf{k}_i c_i}$	$\binom{2}{1}$-OT	$\xleftarrow{\mathbf{r}_i, \mathbf{r}_i + \mathbf{k}_i}$
$\sum_{i=1}^{\kappa} \mathbf{r_i} + c_i \mathbf{k_i}$ $\mathbf{b} \leftarrow \prod_{i=1}^{\kappa} \mathbf{r_i} \mathbf{k_i}^{c_i}$	♻	$\mathbf{k}_0 - (\sum_{i=1}^{\kappa} \mathbf{r}_i)$ $\mathbf{a} \leftarrow \mathbf{k}_0 (\prod_{i=1}^{\kappa} \mathbf{r}_i)^{-1}$
$\xrightarrow{b_i}$ $\xleftarrow{y_i = a_i b_i + e_i}$	OLE	$\xleftarrow{a_i, e_i \leftarrow\$ \mathbb{Z}_q}$
$\mathbf{y} \leftarrow \sum_{i=0}^{n-1} (a_i \cdot b_i + e_i) X^i$	iNTT	$\mathbf{e} \leftarrow \sum_{i=0}^{n-1} e_i X^i$

An OPRF from Spring

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

Client		**Server**
$\mathbf{c} = [c_1, \ldots, c_m] \in \{0,1\}^m$		$\mathbf{K} = [\mathbf{k}_0, \mathbf{k}_1, \ldots, \mathbf{k}_m] \in \{0,1\}^r$
$\xrightarrow{c_i}$ $\xleftarrow{\mathbf{r}_i + \mathbf{k}_i c_i}$	$\binom{2}{1}$-OT	$\xleftarrow{\mathbf{r}_i, \mathbf{r}_i + \mathbf{k}_i}$
$\sum_{i=1}^{\kappa} \mathbf{r_i} + c_i \mathbf{k_i}$ $\mathbf{b} \leftarrow \prod_{i=1}^{\kappa} \mathbf{r_i} \mathbf{k_i}^{c_i}$	♻	$\mathbf{k}_0 - (\sum_{i=1}^{\kappa} \mathbf{r}_i)$ $\mathbf{a} \leftarrow \mathbf{k}_0 (\prod_{i=1}^{\kappa} \mathbf{r}_i)^{-1}$
$\xrightarrow{b_i}$ $\xleftarrow{y_i = a_i b_i + e_i}$	OLE	$\xleftarrow{a_i, e_i \leftarrow\$ \mathbb{Z}_q}$
$\mathbf{y} \leftarrow \sum_{i=0}^{n-1} (a_i \cdot b_i + e_i) X^i$	iNTT	$\mathbf{e} \leftarrow \sum_{i=0}^{n-1} e_i X^i$

An OPRF from Spring

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

Client		**Server**
$\mathbf{c} = [c_1, \ldots, c_m] \in \{0,1\}^m$		$\mathbf{K} = [\mathbf{k}_0, \mathbf{k}_1, \ldots, \mathbf{k}_m] \in \{0,1\}^n$
$\xrightarrow{c_i}$ $\xleftarrow{\mathbf{r}_i + \mathbf{k}_i c_i}$	$\binom{2}{1}$-OT	$\xleftarrow{\mathbf{r}_i, \mathbf{r}_i + \mathbf{k}_i}$
$\sum_{i=1}^{\kappa} \mathbf{r_i} + c_i \mathbf{k_i}$ $\mathbf{b} \leftarrow \prod_{i=1}^{\kappa} \mathbf{r_i} \mathbf{k_i}^{c_i}$	♻	$\mathbf{k}_0 - (\sum_{i=1}^{\kappa} \mathbf{r}_i)$ $\mathbf{a} \leftarrow \mathbf{k}_0 (\prod_{i=1}^{\kappa} \mathbf{r}_i)^{-1}$
$\xrightarrow{b_i}$ $\xleftarrow{y_i = a_i b_i + e_i}$	OLE	$\xleftarrow{a_i, e_i \leftarrow\$ \mathbb{Z}_q}$
$\mathbf{y} \leftarrow \sum_{i=0}^{n-1} (a_i \cdot b_i + e_i) X^i$	iNTT	$\mathbf{e} \leftarrow \sum_{i=0}^{n-1} e_i X^i$
$\xrightarrow{y_i}$ $\xleftarrow{\lfloor y_i - e_i \rceil_p}$	$\binom{q}{1}$-OT	$\xleftarrow[\forall q_i \in \mathbb{Z}_q]{\lfloor (q_i - e_i) \rceil_p}$

An OPRF from Spring

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

Client		**Server**
$\mathbf{c} = [c_1, \ldots, c_m] \in \{0,1\}^m$		$\mathbf{K} = [\mathbf{k}_0, \mathbf{k}_1, \ldots, \mathbf{k}_m] \in \{0,1\}^n$
$\xrightarrow{c_i}$ $\xleftarrow{\mathbf{r}_i + \mathbf{k}_i c_i}$	$\binom{2}{1}$-OT	$\xleftarrow{\mathbf{r}_i, \mathbf{r}_i + \mathbf{k}_i}$
$\sum_{i=1}^{\kappa} \mathbf{r_i} + c_i \mathbf{k_i}$ $\mathbf{b} \leftarrow \prod_{i=1}^{\kappa} \mathbf{r_i} \mathbf{k_i}^{c_i}$	♻	$\mathbf{k}_0 - (\sum_{i=1}^{\kappa} \mathbf{r}_i)$ $\mathbf{a} \leftarrow \mathbf{k}_0 (\prod_{i=1}^{\kappa} \mathbf{r}_i)^{-1}$
$\xrightarrow{b_i}$ $\xleftarrow{y_i = a_i b_i + e_i}$	OLE	$\xleftarrow{a_i, e_i \leftarrow\$ \mathbb{Z}_q}$
$\mathbf{y} \leftarrow \sum_{i=0}^{n-1} (a_i \cdot b_i + e_i) X^i$	iNTT	$\mathbf{e} \leftarrow \sum_{i=0}^{n-1} e_i X^i$ $\mathbf{d} \leftarrow\$ \{0,1\}^n$
$\xrightarrow{y_i}$ $\xleftarrow{\lfloor y_i - e_i \rceil_p \oplus d_i}$	$\binom{q}{1}$-OT	$\xleftarrow{\lfloor (q_i - e_i) \rceil_p \oplus d_i}$ $\forall q_i \in \mathbb{Z}_q$
	$\xleftarrow{\text{BCH}(\mathbf{d})}$	

I am very proud of it

- ✔ proof in the semi-honest model

I am very proud of it

- ✔ proof in the semi-honest model
- two problems make it hard to claim malicious security

Server Attack

- generic attack on Naor-Reingold OPRFs

Server Attack

- generic attack on Naor-Reingold OPRFs
- only works when the server can reason about the OPRF output

Server Attack

- generic attack on Naor-Reingold OPRFs
- only works when the server can reason about the OPRF output
- either because it is revealed later or because the protocol gives information (e.g. whether or not an authentication attempt succeeds.)

Server Attack

- generic attack on Naor-Reingold OPRFs
- only works when the server can reason about the OPRF output
- either because it is revealed later or because the protocol gives information (e.g. whether or not an authentication attempt succeeds.)

Example (Generic Attack on Naor-Reingold OPRFs)

Instead of honestly inputting $\mathbf{k}_i + \mathbf{r}_i$ into the $\binom{2}{1}$-OT, a malicious server may input some $\Delta_i + \mathbf{r_i}$ for some $\Delta_i \neq \mathbf{k}_i$. When the client result is revealed, the server can learn whether or not the ith bit is set.

Server Attack

- generic attack on Naor-Reingold OPRFs
- only works when the server can reason about the OPRF output
- either because it is revealed later or because the protocol gives information (e.g. whether or not an authentication attempt succeeds.)

Example (Generic Attack on Naor-Reingold OPRFs)

Instead of honestly inputting $\mathbf{k}_i + \mathbf{r}_i$ into the $\binom{2}{1}$-OT, a malicious server may input some $\Delta_i + \mathbf{r_i}$ for some $\Delta_i \neq \mathbf{k}_i$. When the client result is revealed, the server can learn whether or not the ith bit is set.

Solution? Verifiable variant of this OPRF.

Server Attack

- generic attack on Naor-Reingold OPRFs
- only works when the server can reason about the OPRF output
- either because it is revealed later or because the protocol gives information (e.g. whether or not an authentication attempt succeeds.)

Example (Generic Attack on Naor-Reingold OPRFs)

Instead of honestly inputting $\mathbf{k}_i + \mathbf{r}_i$ into the $\binom{2}{1}$-OT, a malicious server may input some $\Delta_i + \mathbf{r_i}$ for some $\Delta_i \neq \mathbf{k}_i$. When the client result is revealed, the server can learn whether or not the ith bit is set.

Solution? Verifiable variant of this OPRF. But Naor-Reingold keys are rather large which makes the verification less efficient.

Client Attack

- Leak public key coefficients by faulting the rounding step of the OPRF.

Client Attack

- Leak public key coefficients by faulting the rounding step of the OPRF.
- The key observation is that the rounding result is deterministic and constant for fixed inputs.

Client Attack

- Leak public key coefficients by faulting the rounding step of the OPRF.
- The key observation is that the rounding result is deterministic and constant for fixed inputs.
- Add an offset Δ to the coefficient and leak the exact point where the rounding result flips from zero to one or vice versa.

Client Attack

- Leak public key coefficients by faulting the rounding step of the OPRF.
- The key observation is that the rounding result is deterministic and constant for fixed inputs.
- Add an offset Δ to the coefficient and leak the exact point where the rounding result flips from zero to one or vice versa.
- The attack is mitigated by enforcing consistency between the OLE output and the rounding input so the client cannot compute the offset anymore.

Observation on the modulus

- The BCH attack exploits the rounding bias.

Observation on the modulus

- The BCH attack exploits the rounding bias.
- The CRT attack exploits the linearety.

Observation on the modulus

- The BCH attack exploits the rounding bias.
- The CRT attack exploits the linearety.
- ... combine both to mitigate attack?

Observation on the modulus

- The BCH attack exploits the rounding bias.
- The CRT attack exploits the linearety.
- ... combine both to mitigate attack?
- We suggested this!

Observation on the modulus

- The BCH attack exploits the rounding bias.
- The CRT attack exploits the linearety.
- ... combine both to mitigate attack?
- We suggested this!
  Still leak key offset, but cannot determine actual coefficient

Observation on the modulus

- The BCH attack exploits the rounding bias.
- The CRT attack exploits the linearety.
- ... combine both to mitigate attack?
- We suggested this!
  Still leak key offset, but cannot determine actual coefficient
- Probability of $\frac{1}{2}^n$ of guessing the correct key!

Observation on the modulus

- The BCH attack exploits the rounding bias.
- The CRT attack exploits the linearety.
- ... combine both to mitigate attack?
- We suggested this!
  Still leak key offset, but cannot determine actual coefficient
- Probability of $\frac{1}{2}^n$ of guessing the correct key!
- Cool, but hacky: we leak a lot about the key. The original goal was designing lattice PRFs without ZK.

Let's work it out in the remix

- Leak everything but the MSB?

Let's work it out in the remix

- Leak everything but the MSB? Reduce modulus!

Let's work it out in the remix

- Leak everything but the MSB? Reduce modulus!
- Lattice estimator times out a lot, but $q = 16$ seems secure.

Let's work it out in the remix

- Leak everything but the MSB? Reduce modulus!
- Lattice estimator times out a lot, but $q = 16$ seems secure. Reduce communication by half!

Let's work it out in the remix

- Leak everything but the MSB? Reduce modulus!
- Lattice estimator times out a lot, but $q = 16$ seems secure. Reduce communication by half!
- Would support that the exponential modulus for LWR PRFs is a proof artifact!

Let's work it out in the remix

- Leak everything but the MSB? Reduce modulus!
- Lattice estimator times out a lot, but $q = 16$ seems secure. Reduce communication by half!
- Would support that the exponential modulus for LWR PRFs is a proof artifact!
- Lattice attacks?

Conclusion

- Leap has a proof in the semi-honest model

Conclusion

- Leap has a proof in the semi-honest model
  Some problems to make it malicious secure.

Conclusion

- Leap has a proof in the semi-honest model
  Some problems to make it malicious secure.
- Parameter choice may be sub-optimal.

Conclusion

- Leap has a proof in the semi-honest model
  Some problems to make it malicious secure.
- Parameter choice may be sub-optimal.
- What is the concrete best attack on tiny moduli?

Conclusion

- Leap has a proof in the semi-honest model
  Some problems to make it malicious secure.
- Parameter choice may be sub-optimal.
- What is the concrete best attack on tiny moduli?
- How to prove malicious security with that much leakage and interaction? UC does not work (blind computation).

Acknowledgements

- Thank you, Sonja, Hannah, Sebastian, Reinhard, Shibam, Markus and Christian for your helpful comments on an earlier version of this lecture!
- This work is partially funded by the Digital Europe Program under grant agreement number 101091642 ("QCI-CAT").

Small Moduli, Big Impact: Toward Communication-Efficient Lattice-Based OPRFs

Lena Heimberger

Monash University

1st of December, 2025

References i

Jan-Pieter D'Anvers, Angshuman Karmakar, Sujoy Sinha Roy, Frederik Vercauteren, Jose Maria Bermudo Mera, Michiel Van Beirendonck, and Andrea Basso.
SABER.
Technical report, National Institute of Standards and Technology, 2020.
available at `https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions`.

Michael J. Freedman, Yuval Ishai, Benny Pinkas, and Omer Reingold.
Keyword search and oblivious pseudorandom functions.
pages 303–324, 2005.

References ii

Oded Goldreich, Shafi Goldwasser, and Silvio Micali.
On the cryptographic applications of random functions.
pages 276–288, 1984.

Oded Goldreich, Shafi Goldwasser, and Silvio Micali.
How to construct random functions.
Journal of the ACM, 33(4):792–807, October 1986.

Lena Heimberger, Daniel Kales, Riccardo Lolato, Omid Mir, Sebastian Ramacher, and Christian Rechberger.
Leap: A fast, lattice-based OPRF with application to private set intersection.
pages 254–283, 2025.

Daniel Kales, Christian Rechberger, Thomas Schneider, Matthias Senker, and Christian Weinert.
Mobile private contact discovery at scale.
pages 1447–1464, 2019.

References iii

Moni Naor and Omer Reingold.
Number-theoretic constructions of efficient pseudo-random functions.
Journal of the ACM, 51(2):231–262, March 2004.